

Konstantin Lucius Regenhardt

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Bachelorarbeit eingereicht im Rahmen der Bachelorprüfung
im Studiengang *Bachelor of Science Umweltinformatik*
am Department Fachbereich 2
der Fakultät Technik und Informatik
der Hochschule für Technik und Wirtschaft Berlin

Betreuender Prüfer: Prof. Dr. Jochen Wittmann
Zweitgutachter: Prof. Dr. Zaphod Beeblebrox

Eingereicht am: 01.08.2026

Konstantin Lucius Regenhardt

Thema der Arbeit

Entwurf und Evaluation eines automatisierten Migrationswerkzeugs von Katalon-Groovy-Tests zu Selenium-Pytest

Stichworte

Kurzzusammenfassung

In der Softwareentwicklung bieten proprietäre Low-Code-Plattformen wie Katalon einen schnellen Einstieg in die Testautomatisierung. Mit steigender Projektkomplexität führen diese jedoch häufig zu einem massiven Kostenanstieg durch unflexible Lizenzmodelle für fortgeschrittene Funktionen. Da die Testlogik und Datenobjekte in proprietären Formaten gespeichert sind, stehen Nutzer vor dem Dilemma, entweder hohe Gebühren zu zahlen oder den Verlust der bisherigen Entwicklungsarbeit bei einem Plattformwechsel hinzunehmen. Ziel dieser Bachelorarbeit ist die Entwicklung eines programmatischen Migrationspfades von Katalon zu einer Open-Source-Alternative, geschrieben in Selenium. Dabei soll die manuelle Neuerstellung der Test-Suites durch eine automatisierte Transformations-Pipeline ersetzt werden, um die bisherige Arbeit nicht zu verlieren. Es wird ein Migrations-Algorithmus entworfen, der auf Regular Expressions und Python-Methoden basiert. Dieser scannt die proprietären Strukturen, übersetzt die bestehende Testlogik von Groovy nach Python und bildet die internen Datenobjekte sowie Referenzdateien auf eine neue, Open-Source Projektstruktur im Selenium-Framework ab. Das Ergebnis ist ein funktionsfähiger Prototyp eines Migrations-Tools. Dieses Tool ermöglicht den Transfer von Test-Suites in ein erweiterbares Open-Source-Projekt. Dabei bleibt die Integrität der Tests gewahrt und die Abhängigkeit von Lizenzgebühren wird vollständig eliminiert. Die Arbeit zeigt auf, wie durch automatisierte Code-Migration und -Transformation der Wechsel von proprietären „Low-Code“ zu Open-Source „Pro-Code“ gelingen kann. Die Ergebnisse bieten Nutzern eine gute Möglichkeit für den Ausstieg aus proprietären Test-Ökosystemen.

Konstantin Lucius Regenhardt

Title of Thesis

From Proprietary Test Automation to Open Execution: Design and Evaluation of a Katalon-to-Selenium/Pytest Migration Tool

Keywords

Abstract

In software development, proprietary low-code platforms like *Katalon* offer a fast initial entry point into test automation. However, as project complexity scales, these systems frequently lead to a massive escalation of costs due to inflexible licensing frameworks. This thesis addresses this „Vendor Lock-In“ problem by establishing an automated transformation pipeline that migrates test suites from closed Katalon structures into an open-source, flexible framework written in Python using *Selenium*.

A core migration algorithm based on Python execution logic and Regular Expressions (Regex) handles the automated parsing of proprietary formats, translations of test scripts from *Groovy* to *Python*, and mapping of object repositories. The evaluation showcases a fully functional prototype that successfully eliminates license fee dependencies while completely preserving test suite runtime integrity.

IDE – Integrated Development Environment: Eine Software Anwendung, die Entwicklern eine umfassende Umgebung für die Softwareentwicklung bietet. 5, 5, 5

UI – User Interface: Die Benutzerschnittstelle einer Anwendung, ueber die Nutzer mit dem System interagieren. 5

Inhaltsverzeichnis

Abbildungsverzeichnis	V
Tabellenverzeichnis	VI
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Ziel & Aufbau der Arbeit	2
2 Grundlagen	2
2.1 Automatisiertes Testen	2
2.2 Vendor Lock-in bei proprietären Plattformen	3
2.3 Low-Code-Plattformen im Testbereich	3
3 Strukturanalyse	4
3.1 Katalon Projektstruktur	4
3.1.1 Test Cases & Variablen	4
3.1.2 Object Repository	4
3.1.3 Globale Variablen & Profile	4
3.1.4 Testdaten & Einbindung	4
3.1.5 Custom Keywords	4
3.1.6 Verschachtelungen in der Katalon Struktur	5
3.1.7 Verschachtelung der Custom Keywords	6
3.2 Selenium/Pytest Projektstruktur	6
3.2.1 Testskripte & Variablen	6
3.2.2 Testdaten & Einbindung	6
3.2.3 Pytest Fixtures & Konfiguration	7
4 Conception of the Migration Pipeline	7
5 Implementation of the Prototype	8
6 Evaluation & Results	8
7 Discussion & Future Work	8
8 Conclusion / Zusammenfassung	8
9 Appendix / Anhang	9
10 Declaration of Originality / Eigenständigkeitserklärung	9
Quellenverzeichnis	10
Erklärung zur selbstständigen Bearbeitung	11

Abbildungsverzeichnis

Abbildung 1	Comparative overview of proprietary Low-Code platform scaling vs. Open-Source alternatives. (Source: Own work)	7
Abbildung 2	System architecture of the automated Groovy-to-Python Regex transformation engine. (Source: Adapted from Smith et al., 2024, p. 12) . . .	7

Tabellenverzeichnis

Tabelle 1 Runtime performance and execution velocity comparisons between the legacy system and the migrated prototype. (Source: Compiled from test logs) 8

1 Einleitung

1.1 Motivation

Automatisiertes Testen ist eine zentrale Komponente moderner Software-Qualitätssicherung. In vielen Teams werden Testautomatisierungs-Frameworks eingesetzt, um Regressionstests zu beschleunigen und das Vertrauen in Veröffentlichungen zu stärken[1]. Anfangs nutzte unser Team Katalon Studio als IDE um neben der Entwicklung einer internen Geschäftssoftware auch eine grundlegende Testinfrastruktur aufzubauen. Im Laufe der Zeit wuchs die Infrastruktur stark an und umspannte den Großteil der Software.

Da sich die Anforderungen des Projekts weiterentwickelten, benötigte das Team fortgeschrittenere Automatisierungs-Techniken, wie automatisiertes Berechnen von Datums-Einträgen und eine tiefere Kontrolle über die Testausführung und das Debugging. In dieser Phase traten erste Blockaden auf: Erweiterte Funktionen, die im Programmieren als elementar gesehen werden, erforderten zusätzliche Lizenzen, was erhebliche Kosten verursachte und die Skalierung der Infrastruktur einschränkte. Dies kristallisierte sich schnell als ein blockierendes Problem heraus da die Preise in keinem Verhältnis zu den Leistungen standen und über längere Zeit nicht tragbar waren. Als Lösungsansatz wurde eine Migration zu einem Open-Source Projekt vorgeschlagen.

Angesichts der erheblichen bestehenden Investitionen in eine Katalon-basierte Test-Umgebung war ein vollständiges Neuschreiben der aller gesammelten Tests von Grund auf, nicht machbar. Deswegen wurde ein Migrationsansatz als PoC untersucht, um die zuvor erstellten Tests und alles Zugehörige zu erhalten und gleichzeitig den Übergang zu einer offenen und erweiterbaren Open-Source Struktur zu ermöglichen.

1.2 Problemstellung

Das Kernproblem das in dieser Arbeit behandelt wird, ist die Frage wie ein bestehendes Katalon-Projekt beim Wechsel in eine lizenzunabhängige Open-Source Umgebung beibehalten und wiederverwendet werden kann. Eine manuelle Migration von Katalon-Tests zu Selenium/Pytest ist extrem zeitaufwendig und bei einer großen Testmenge praktisch unerreichbar. Das liegt daran, dass neben der reinen Skriptübersetzung eine Menge an zusätzlichen Abhängigkeiten und Konfigurationen berücksichtigt werden müssen. Ein Katalon-Projekt besteht nicht nur aus Testskripten, sondern auch aus einer Vielzahl von Assets wie Objekt-Repositories, Variablen-Definitionen, globalen Profilen und Datenabhängigkeiten. Diese sind jeweils in proprietären Formaten gespeichert und müssten manuell „ausgegraben“ und zusammengeführt werden. Eine manuelle Migration würde daher nicht nur die Übersetzung von Groovy nach Python erfordern, sondern auch die Rekonstruktion der gesamten Projektstruktur und die Anpassung an die Open-Source-Frameworks.

Die technische Herausforderung liegt nicht nur in der syntaktischen Konvertierung des Testcodes, sondern in der End-to-End-Transformation eines Testautomatisierungsprojekts in ein ausführbares und skalierbares alternatives Format.

1.3 Ziel & Aufbau der Arbeit

Das Ziel dieser Arbeit ist es, einen automatisierten Migrator zu entwerfen, zu implementieren und zu evaluieren, der Katalon-basierte Testprojekte in eine Python-Selenium/Pytest-Projektstruktur konvertiert.

Um dieses Ziel zu erreichen wird in dieser Arbeit zuerst werden die Struktur eines Katalon-Projektes und die eines Python-Selenium/Pytest-Projektes analysiert. Dann wird Stück für Stück ein Migrationsalgorithmus entworfen, der die proprietären Formate des einen in offen nutzbare Formate des anderen überführt. Anschließend wird ein Prototyp implementiert, der die Transformation automatisiert und die Integrität der Tests sicherstellt. Abschließend werden die Ergebnisse evaluiert und diskutiert, um die Effektivität des Migrationsprozesses zu bewerten.

2 Grundlagen

2.1 Automatisiertes Testen

Softwaretests sind ein grundlegender Bestandteil der Qualitätssicherung in der Softwareentwicklung. Ihr Ziel ist es, sicherzustellen, dass ein System korrekt funktioniert und definierte Anforderungen erfüllt. Dabei unterscheidet man grob zwischen manuellen und automatisierten Tests.

Bei manuellen Tests führt eine Person die Testschritte selbst aus und bewertet das Ergebnis. Dieser Ansatz ist bei kleinen, seltenen oder testhaften Überprüfungen praktikabel, skaliert jedoch schlecht: Mit wachsender Softwarekomplexität steigt der Zeitaufwand für manuelle Regressionstests exponentiell.

Automatisiertes Testen bezeichnet das maschinengestützte Ausführen von Testfällen anhand von Testskripten. Die Skripte definieren Eingaben, Aktionen und erwartete Ergebnisse. Wesentliche Vorteile automatisierter Tests sind[2]:

- **Wiederholbarkeit:** Dieselben Tests können beliebig oft und konsistent ausgeführt werden.
- **Geschwindigkeit:** Große Test-Suiten lassen sich in kurzer Zeit ausführen.
- **Regressionssicherheit:** Nach Codeänderungen kann schnell geprüft werden, ob bestehende Funktionalität noch korrekt arbeitet.
- **CI/CD-Integration:** Automatisierte Tests können direkt in Deployment-Pipelines eingebunden werden.

Ein häufig eingesetztes Verfahren ist der **End-to-End-Test** (E2E-Test), bei dem eine Anwendung aus Nutzerperspektive durch die Benutzeroberfläche getestet wird. Werkzeuge wie die Selenium WebDriver API ermöglichen solche Tests für webbasierte Anwendungen, indem sie einen Browser programmatisch steuern.

2.2 Vendor Lock-in bei proprietären Plattformen

Vendor Lock-in beschreibt die Abhängigkeit eines Nutzers oder einer Organisation von einem bestimmten Anbieter, sodass ein Wechsel zu einer Alternative mit erheblichem Aufwand oder hohen Kosten verbunden ist[3]. Diese Abhängigkeit entsteht häufig durch proprietäre Dateiformate, plattformspezifische Programmiersprachen oder APIs, die außerhalb des jeweiligen Ökosystems nicht verwendet werden können.

Im Bereich der Testautomatisierung zeigt sich Vendor Lock-in beispielsweise dann, wenn:

- Testskripte in einer plattformeigenen Skriptsprache verfasst sind und nicht ohne Weiteres auf andere Frameworks übertragen werden können,
- interne Datenstrukturen wie Objekt-Repositorys oder Profil-Konfigurationen in proprietären Formaten gespeichert sind,
- bestimmte Funktionen nur in kostenpflichtigen Lizenzstufen verfügbar sind.

Die Konsequenz ist, dass Teams entweder steigende Lizenzkosten akzeptieren oder bei einem Plattformwechsel einen erheblichen Teil ihrer bisherigen Arbeit verlieren. Für das in dieser Arbeit beschriebene Projekt manifestierte sich dieses Problem konkret: Grundlegende Entwicklungsfunktionen wie das Debugging über die CLI erforderten teurere Lizenzpakete, was die langfristige Nutzung der Plattform unwirtschaftlich machte.

2.3 Low-Code-Plattformen im Testbereich

Low-Code-Plattformen sind Entwicklungsumgebungen, die komplexe technische Operationen durch grafische Oberflächen und vorgefertigte Bausteine abstrahieren. Im Bereich der Testautomatisierung bieten sie einen niedrighschwelligen Einstieg: Tests können über eine GUI aufgezeichnet oder aus einem Katalog von vordefinierten Aktionen zusammengestellt werden, ohne tiefgehende Programmierkenntnisse vorauszusetzen.

Diese Eigenschaft macht Low-Code-Tools attraktiv für Teams in der frühen Phase der Testautomatisierung. Mit wachsender Anforderungskomplexität stoßen sie jedoch an Grenzen[4]:

- Individuelle Logik, die über vorgefertigte Bausteine hinausgeht, ist schwer oder gar nicht umsetzbar.
- Die Testlogik ist eng an die Plattform gebunden und schlecht in externe Versionskontrollsysteme integrierbar.
- Skalierbarkeit und Anpassbarkeit sind durch das Plattformmodell begrenzt.

Im Gegensatz dazu stehen sogenannte **Pro-Code**-Ansätze, bei denen Tests vollständig in einer allgemeinen Programmiersprache wie Python geschrieben werden. Frameworks wie Selenium und Pytest bieten dabei maximale Flexibilität, erfordern aber entsprechende Programmierkenntnisse. Der Übergang von Low-Code zu Pro-Code ist inhaltlich des Kerns dieser Arbeit.

3 Strukturanalyse

3.1 Katalon Projektstruktur

Ein Katalon Studio Projekt besteht, aus dem Blickwinkel des IDE-Benutzers, unter Anderem aus Tests, einem „Object Repository“, globalen Variablen in Profilen, test eigenen Variablen, eingebundenen Test-Daten. Diese werden in einer hierarchischen Ordnerstruktur gespeichert, die von der IDE verwaltet wird.

3.1.1 Test Cases & Variablen

Die Testskripte in Katalon sind in Groovy geschrieben, einer dynamischen Sprache, die auf der Java Virtual Machine (JVM) läuft. Sie werden mit einer Vielzahl von eingebauten Keywords und Funktionen geliefert, die speziell für Testautomatisierung entwickelt wurden. Variablen können sowohl global als auch lokal definiert werden, wobei globale Variablen in Profilen gespeichert sind und in allen Tests zugänglich sind.

3.1.2 Object Repository

Das Object Repository ist ein zentrales Element in der Katalon Umgebung, das das Speichern von UI-Elementen ermöglicht. Es speichert die Eigenschaften von HTML-Elementen, die während der Testausführung verwendet werden. Dafür kann der Benutzer die Elemente mit einem Tool das Katalon Studio bereitstellt auf der Webseite auswählen und im Object Repository generieren lassen. Diese Elemente können in verschiedenen Tests wiederverwendet werden, was die Wartung und Skalierbarkeit der Tests erleichtert.

3.1.3 Globale Variablen & Profile

Jede/r User/in kann in Katalon eigene globale Variablen definieren, die in verschiedenen Test Cases verwendet werden können. Diese Variablen werden in Profilen gespeichert, welche unterschiedliche Benutzende oder Umgebungen repräsentieren können. Profile ermöglichen es, Tests in verschiedenen Konfigurationen auszuführen, ohne den Testcode selbst ändern zu müssen. Hat ein Profil die Variable „URL“ mit dem Wert „https://staging.example.com“ und ein anderes Profil die gleiche Variable mit dem Wert „https://production.example.com“, kann der Test in beiden Umgebungen ausgeführt werden, indem einfach das entsprechende Profil ausgewählt wird.

3.1.4 Testdaten & Einbindung

Katalon kann Testdaten aus verschiedenen Quellen, wie Excel-Dateien, CSV-Dateien oder Datenbanken einbinden. Diese Testdaten können in den Testskripten referenziert werden, um die Tests mit unterschiedlichen Eingabewerten auszuführen.

3.1.5 Custom Keywords

Benutzer/Innen können eigene Methoden, sogenannte „Custom Keywords“ erstellen, die in mehreren Test Cases wiederverwendet werden können. Diese Keywords werden in Groovy geschrieben und ermöglichen es, komplexere Logik für Tests zu nutzen und diese wiederverwendbar zu machen.

3.1.6 Verschachtelungen in der Katalon Struktur

Alle Dateien und Ordner die man in der Katalon Integrated Development Environment (IDE) erstellt, werden durch den Viewport übersichtlich dargestellt. Sobald man sich die Ordnerstruktur eines Katalon-Projekts jedoch außerhalb der IDE anschaut, erkennt man, dass die Testskripte nicht nur eine einzelne Datei sind, wie im Programmieren üblich.

3.1.6.1 Verschachtelung der Test Cases

Ein „Test Case“ besteht in Katalon aus mehreren Dateien, die zusammenarbeiten. Folgt man der Katalon-Ordnerstruktur im Explorer unter „Test Cases“, so wie sie in der IDE zu sehen ist landet man bei einer .tc-Datei. Öffnet man diese, erkennt man schnell, dass es sich dabei um eine umbenannte .xml-Datei handelt. Sie enthält Meta-Daten, wie eine Beschreibung, den Testnamen, Tags, Kommentare, eine GUID und die eigentlichen Werte der Test-eigenen Variablen. Es gibt keine offensichtliche Referenz auf die eigentliche Testlogik. Diese ist in einer weiteren Datei gespeichert. Um sie zu finden muss man den Ordner „Scripts“ öffnen, dann den Pfad des „Test Cases“ spiegeln. Dort befindet sich ein Ordner der den Testnamen trägt und dieser enthält dann eine .groovy-Datei, die den Namen „Script“ kombiniert mit einer zufälligen Nummer trägt. Die Nummer steht in keiner Verbindung zu der .tc-Datei. In dieser .groovy-Datei befindet sich dann die eigentliche Testlogik. Greift der Test auf eine Variable zu, wird im Skript der „rohe“ Name der Variable ausgeschrieben. „Roh“ bedeutet, dass der Name im Code keinen Datentyp hat und auf nichts im Test verweist. Die Variable wird im Test weder deklariert, noch initialisiert.

3.1.6.2 Verschachtelung der Globalen Variablen

Wenn der Test eine Globale Variable verwendet, zeigt sich dies im Code als „GlobalVariable.VARIABLENNAME“. Der Wert der Variable ist nicht im Skript zu finden sondern in der Profil-Datei mit der Endung .gbl, die im Ordner „Profiles“ gespeichert ist. Diese Datei ist ebenfalls eine .xml-Datei und enthält die Variablen als Eintrag in einer „GlobalVariableEntity“ - ein Eintrag der die Variable beschreibt. Diese Einträge enthalten den tatsächlichen Wert und Meta-Daten, bestehend aus einer Beschreibung, dem Variablennamen, dem Datentyp und einem Boolean der beschreibt ob der Wert „protected“ ist. Ist er „protected“, wird der Wert als sensibel behandelt und in der User Interface (UI) und bei der Bearbeitung nur mit dem Stern-Charakter maskiert angezeigt.

3.1.6.3 Verschachtelung der Testdaten

Sobald man mit Katalon Studio ein Dateiformat mit Daten einbinden möchte, erstellt Katalon Studio eine .dat-Datei im Ordner „Data Files“ die den gleichen Namen wie die eingebundene Datei trägt. Die Datei enthält ebenfalls XML-Strukturen, nur ein paar Meta-Daten enthalten. Unter anderem den Pfad zur eingebundenen Datei, die Dateiart, ggf. die Trennzeichen und ob der Pfad Projektintern oder -extern ist. Die eigentlichen Daten sind nicht in der .dat-Datei enthalten, sondern in der eingebundenen Datei.

3.1.7 Verschachtelung der Custom Keywords

Der Großteil der „Custom Keywords“-Logik wird im Ordner „Keywords“ und selbst erstellten Unterordnern gespeichert, und ist in Groovy in der Form „KlassenName.groovy“ geschrieben. Diese Skripte enthalten die eigentlichen Klassen und Methoden die in den Tests aufgerufen werden. Im Ordner „Libs“ gibt es die „CustomKeywords.groovy“-Datei. Diese definiert statische Weiterleitungen mit dem gleichen Namen wie die Klassen im Ordner „Keywords“. Diese Methoden rufen die eigentlichen Methoden in den Klassen auf und machen sie dadurch in jedem Test aufrufbar.

3.2 Selenium/Pytest Projektstruktur

Ein Projekt, das Selenium und Pytest für die Testautomatisierung nutzt, ist im Gegensatz zu Katalon nicht an eine proprietäre Projektstruktur gebunden. Pytest erwartet nur eine nachvollziehbare Dateioorganisation durch Konventionen, gültige Python-Module und eine Konfiguration im Projektwurzelverzeichnis, der „root“, in Form einer pyproject.toml oder pytest.ini[5]. In einem reinen Testprojekt enthält das Repository dabei keinen Anwendungscode, sondern ausschließlich Testfälle, gemeinsam genutzte Hilfsmodule, Konfigurationsdateien und Testdaten. Typischerweise existiert ein zentrales „tests“-Verzeichnis, das bei wachsender Projektgröße weiter in Teilbereiche der zu testenden Anwendung gegliedert wird, beispielsweise in UI-Tests und API-Tests[6]. Das aus Python-Paketprojekten bekannte src-Layout ist in diesem Fall nicht zwingend erforderlich. Laut Python Packaging User Guide dient das src-Layout vor allem dazu, importierbaren Anwendungscode klar von der Projekt-root zu trennen[7].

3.2.1 Testskripte & Variablen

Die eigentlichen Testfälle folgen den von Pytest vorgesehenen Namenskonventionen wie test_NAME.py oder NAME_test.py[6]. Anders als bei Katalon besteht ein Testfall dabei in der Regel aus genau einer Python-Datei, in der die Testlogik direkt lesbar ist. Wiederverwendbare Browser-Interaktionen, Element-Selektoren oder Methoden können in separate Hilfsmodule ausgelagert werden. Diese heißen oft „Helper“- oder „Utility“-Funktionen.

Variablen werden in einem solchen Projekt nicht über proprietäre Profil- oder Metadateien verwaltet, sondern über normale Python-Programmierstrukturen. Typisch sind Parametrisierungen in den Testfunktionen, Rückgabewerte von Fixtures, Umgebungsvariablen oder externe Konfigurationsdateien. Dadurch bleibt nachvollziehbar, woher ein Wert stammt und an welcher Stelle er in den Test eingebunden wird.

3.2.2 Testdaten & Einbindung

Testdaten werden häufig in einem eigenen Verzeichnis wie z. B. „data“, „resources“ oder „testdata“ abgelegt und dann bei Bedarf direkt von den Tests oder von Hilfsmodulen eingelesen. Dabei kann es sich beispielsweise um JSON-, CSV-, XML- oder Excel-Dateien handeln, jedoch ist praktisch jedes Format nutzbar. Im Unterschied zu Katalon entsteht hierbei keine zusätzliche proprietäre Metadatei, die nur einen Verweis auf die eigentliche Datenquelle enthält.

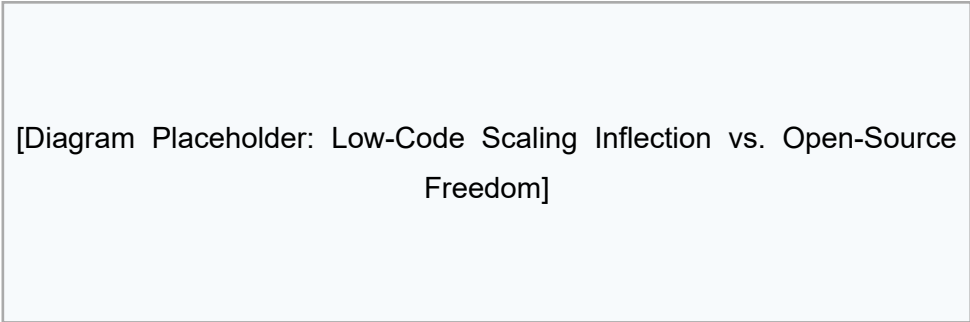
Stattdessen sind Dateipfade, Ladeverhalten und Weiterverarbeitung unmittelbar im Python-Code oder in den zugehörigen Hilfsfunktionen sichtbar.

3.2.3 Pytest Fixtures & Konfiguration

Eine zentrale Rolle in Pytest nimmt die Datei `conftest.py` ein. Die dort definierten „Fixtures“, sind wiederverwendbare Testkomponenten und stehen allen Tests automatisch zur Verfügung und müssen nicht explizit importiert werden[8]. Auf diese Weise lassen sich gemeinsam genutzte Setups, etwa die Erstellung eines WebDriver-Objekts, Logins, temporäre Verzeichnisse oder standardisierte Testdaten, an einer gemeinsamen Stelle sammeln.

Die allgemeine Projektkonfiguration liegt typischerweise im `root`-Verzeichnis des Repositories. Dort können unter anderem Testpfade, Standardoptionen für den Testlauf oder Marker definiert werden[5].

Für ein reines Selenium/Pytest-Testprojekt ergibt sich damit eine vergleichsweise schlanke Struktur aus Testskripten, gemeinsam genutzten Fixtures, optionalen Hilfsmodulen und separaten Testdaten. Im Vergleich zur Katalon-Struktur sind sowohl die Pfade als auch die Abhängigkeiten zwischen Testlogik, Konfiguration und Daten deutlich direkter ablesbar.



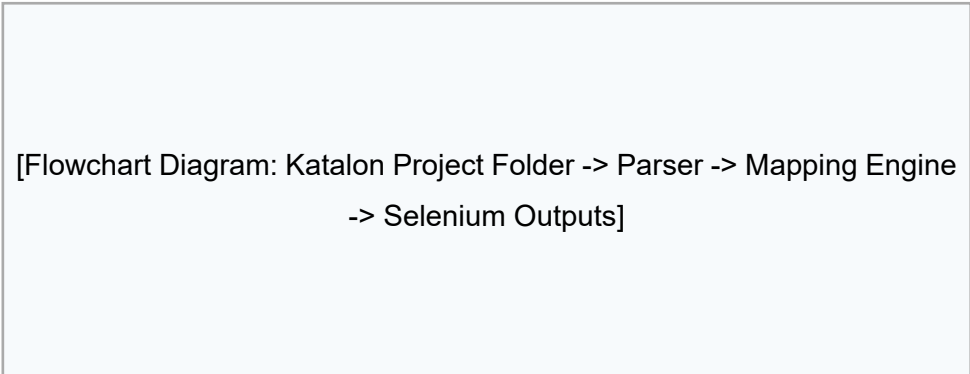
[Diagram Placeholder: Low-Code Scaling Inflection vs. Open-Source Freedom]

Abbildung 1: Comparative overview of proprietary Low-Code platform scaling vs. Open-Source alternatives. (Source: Own work)

As shown in Abbildung 1, the breaking curve represents technical licensing barriers.

4 Conception of the Migration Pipeline

This section details your architectural design principles.



[Flowchart Diagram: Katalon Project Folder -> Parser -> Mapping Engine -> Selenium Outputs]

Abbildung 2: System architecture of the automated Groovy-to-Python Regex transformation engine. (Source: Adapted from Smith et al., 2024, p. 12)

The technical pipeline architecture visualized above in Abbildung 2 details the precise structural transformation layers.

5 Implementation of the Prototype

Detail the technical specifications of your written scripts and matching regex engine loops here.

6 Evaluation & Results

Test Suite Key	Katalon Execution (s)	Selenium Execution (s)
TS-01: Core Authentication	14.2	11.5
TS-02: Inventory Checkout	32.8	24.1
TS-03: Reporting Dashboard	45.1	41.9

Tabelle 1: Runtime performance and execution velocity comparisons between the legacy system and the migrated prototype. (Source: Compiled from test logs)

The validation metrics recorded inside Tabelle 1 show the functional performance of direct WebDriver implementation.

7 Discussion & Future Work

[Evaluate technical constraints, limitations, and future scalability to frameworks like Playwright or Cypress.]

8 Conclusion / Zusammenfassung

[Summarize final project milestones.]

9 Appendix / Anhang

[Attach sample XML files and extracted parsed code structures here.]

10 Declaration of Originality / Eigenständigkeitserklärung

I hereby declare that:

- I have written this academic thesis independently and without any unauthorized assistance,
- I have not used any sources or aids other than those explicitly specified,
- I have clearly marked all passages taken verbatim or in spirit from the referenced sources as citations,
- This work has not been submitted in the same or a similar form to any other examination authority.

.....

.....

Berlin, Date

Konstantin Regenhardt

Quellenverzeichnis

- [1] Katalon, Inc., „Katalon | The AI Platform for Software Quality“. Zugegriffen: 21. Juli 2026. [Online]. Verfügbar unter: <https://katalon.com/>
- [2] International Software Testing Qualifications Board, „ISTQB Glossary“. Zugegriffen: 22. Juli 2026. [Online]. Verfügbar unter: <https://glossary.istqb.org/>
- [3] C. Shapiro und H. R. Varian, *Information Rules: A Strategic Guide to the Network Economy*. Boston: Harvard Business School Press, 1998.
- [4] A. Sahay, A. Indamutsa, D. Di Ruscio, und A. Pierantonio, „Supporting the Understanding and Comparison of Low-Code Development Platforms“, in *Proceedings of the 46th Euro-micro Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2020, S. 171–178. doi: 10.1109/SEAA51224.2020.00036.
- [5] pytest development team, „Pytest Documentation: Configuration“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/reference/customize.html>
- [6] pytest development team, „Pytest Documentation: Good Integration Practices“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/explanation/goodpractices.html>
- [7] Python Packaging Authority, „Python Packaging User Guide: src layout vs flat layout“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/>
- [8] pytest development team, „Pytest Documentation: Fixtures Reference“. Zugegriffen: 24. Juli 2026. [Online]. Verfügbar unter: <https://docs.pytest.org/en/stable/reference/fixtures.html>

Erklärung zur selbstständigen Bearbeitung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne fremde Hilfe selbständig verfasst und nur die angegebenen Hilfsmittel benutzt habe. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht.

Ort

Datum

Unterschrift im Original